

Work Instruction: Standard Git Workflow

1. Purpose

To ensure everyone works with Git in a consistent and standardized way when contributing to shared repositories.

2. Scope

This applies to all developers contributing to shared projects hosted on Git platforms (e.g. GitHub, GitLab, Bitbucket) using a fork + feature branch + pull request workflow.

3. Prerequisites

1. Git installed and configured.
2. Access to the main (upstream) repository.
3. An account on the Git hosting platform (e.g. GitHub).

4. Standard Workflow Overview

(1st time)

1. Fork the main repository.
2. Clone your fork to your local machine.

iterative: 3. Create a new branch for your change. 4. Commit and push your changes to your branch on your fork. 5. Create a Pull Request (PR) from your branch to the main repository. 6. Review & approvals. 7. Merge the PR into the main repository. 8. Sync your fork with the main repository, merge back main into your local branch if needed, or delete it.

5. Detailed Steps

5.1 Forking the main repository

1. Go to the main repository in the web UI (e.g. GitHub).
2. Click Fork.
3. Choose your personal or team account as the destination.
4. Wait until the fork is created.

Result: You now have your own copy of the repository (your fork).

5.1(b) Alternatively: Clone your fork locally

Replace with your actual fork URL:

```
git clone https://github.com/<your-username>/<repo-name>.git
cd <repo-name>
```

5.2 Add the main repository as "upstream" (once per fork)

Replace with main repository URL

```
git remote add upstream https://github.com/<org-or-owner>/<repo-name>.git
```

5.3 Verify remotes

```
git remote -v
```

You should see something like:

```
origin: your fork
upstream: main repository
```

5.4 Create a new branch for your change

Always create a separate branch for each feature or fix. First, update your local main branch:

```
git checkout main
```

```
git pull upstream main
```

Create a new branch from main with Branch naming convention (example):

```
<your-name>-Branch-<short-description>
```

Examples:

```
ChangIkJoong-game-foundation
```

```
ChangIkJoong-bugfix-template
```

```
Nadir-bugfix-login
```

```
Janna-docs-getting-started
```

Full Command Example

```
git checkout -b ChangIkJoong-Branch-Control-Update
```

Result: You are now on a feature branch where you can safely make changes.

5.5 Make changes, commit, and push

Edit files as needed. Either commit and push locally:

Check status:

```
git status
```

Stage changes:

```
git add <file1> <file2>
```

or all changes:

```
git add .
```

Commit with a clear message:

```
git commit -m "Add basic game player movement"
```

Push the branch to your fork:

```
git push origin ChangIkJoong-branch-game-foundation
```

5.5(b) Via the IDE (EASIER OPTION)

This part is easier using the IDE itself (personal opinion).

You can also like above do in the IDE (commit & push option):

```
Add basic game player movement
```

```
Added the basic player movement along with a bug-fix for the  
graphical componenet as this was related to the original update.
```

However, please adhere to the standard - Commit message guideline:

1. Use imperative present tense (e.g. Add, Fix, Update).
2. Be specific about what changed.
3. Always have a headline starting with a capital letter
4. Have the bread text below with 2 newlines.

5.6 Create a Pull Request (PR) to the main repository

Go to your fork in the web UI.

You'll see a prompt like "Compare & pull request" for your recently pushed branch – click it.

If not visible, manually start a new pull request and select:

Base repository: main repository (e.g. org/repo)

Base branch: usually main

Head repository: your fork

Head branch: your branch (e.g. ChangIkJoong-branch-game-foundation)

Fill out PR details:

Title: short and clear

Example: Add game foundation and basic player controls

Description:

What you changed

Why you changed it

How to test it

Assign: Relevant reviewers

Labels, milestone, and linked issues (if applicable)

Result: Your changes are now proposed to the main repository and visible to the team.

5.7(a) Review, feedback, and updates

Reviewers will comment and/or request changes.

To update the PR:

Make changes locally on the same branch.

Commit and push again:

```
git add .  
git commit -m "Address review comments: refactor player movement"  
git push origin ChangIkJoong-branch-game-foundation
```

The PR updates automatically with new commits.

5.7(a) In GitHub

1. Reviewers will comment and/or request changes.
2. To update the PR:
 - Deal with merge conflicts, etc. in the github IDE.

Commit and push / merge to main branch.

5.8 Merge the PR

Once:

All required checks (CI, tests) pass, and

The required number of approvals is received

Then:

Use the web UI to merge:

Preferred strategy (example recommendation): Squash and merge

Keeps the main branch history clean and compact.

Confirm the merge.

Result: Your changes are now part of the main repository's main branch.

5.9 Sync your local repo and fork after merge

After your PR is merged:

Switch to main

```
git checkout main
```

Pull latest changes from upstream (main repo)

```
git pull upstream main
```

Push updated main to your fork

```
git push origin main
```

You can now delete the feature branch:

Locally

```
git branch -d ChangIkJoong-branch-game-foundation
```

On remote (your fork)

```
git push origin --delete ChangIkJoong-branch-game-foundation
```

6. Rules & Best Practices Summary

Always work on a branch, never directly on main.

One branch per change/feature/bugfix.

Use descriptive branch names and commit messages.

Always create a Pull Request to the main repository for review.

Do not merge your own PR without required approvals (follow team rules).

Keep your fork in sync regularly with the main repository.